last name, and date of birth needs three attributes, each of which is a field that corresponds to the stored data type. These attributes serve a dual purpose: they both define the data type of the column in the corresponding database table, and they are used to access the content of the field during a query. A `first_name` attribute with the field type `CharField` both states that the first name gets stored as a character field in the database and is the access point for getting a musician's first name when you query the database.

To represent a musician, you create a `Musician` class and map three fields for the first name, last name, and date of birth. Unlike Python, in which strings can have an arbitrary length, many relational databases want you to be specific about string sizes. The `CharField` requires a `max_length` argument to indicate the size of the column. There are text fields that don't have this limit, but their storage tends to be inefficient;the best practice is to use the `CharField` for fields like names. Your first and last name fields are both `CharField`, while the date of birth gets stored using a `DateField`.

> **How much space?**
>
> Deciding how much is a reasonable size for any field is somewhat based on experience (read: guesswork). In the old days, using 10 characters for a first name was quite common. Christopher, for example, has 11 characters. I used to frequently receive mail addressed to *Christophe*. Names are culture specific, and making assumptions about them can get messy quickly. With disk space being cheap now, being more generous with your field lengths is achievable.

To define your first `Model` class, edit RiffMates/bands/models.py, and change it to look like the following listing.

Listing 4.2 Musician ORM Model class

```
# RiffMates/bands/models.py
from django.db import models

class Musician(models.Model):
    first_name = models.CharField(max_length=50)
    last_name = models.CharField(max_length=50)
    birth = models.DateField()
```

Defining the model is only the first step. Now, you need to create the corresponding table in the database. To map your `Musicians` class to a table, the ORM needs to execute some SQL on the database. Django abstracts this SQL using some Python called a *migration script*. As you make changes to your ORM models, Django tracks the changes and creates new versions of the migration scripts. You then apply the migration scripts to the database. Both the creation and running are done through management commands. The mapping for the `Musicians` model to the table created in the database after the migration gets performed is shown in figure 4.3.